Hoja de grabación — qué mostrar, qué decir y qué ejecutar

> **Plan de rescate: 11:56 en total.** La presentación (P0) ya está grabada. C-03 y Skill bajan a 1:00 cada uno; P6–P9 quedan iguales. El demo arranca en **8:56**, dentro de los primeros 10 minutos: el evaluador ve el sistema funcionando aunque el video siga un poco más.

Timeline de rescate

Tramo	Tiempo	Acumulado
P0–P5 (con C-03 y Skill en 1:00)	7:51	7:51
P6 · MCP	0:45	8:36
P7 · Tests	0:20	**8:56** (arranca el demo)
P8 · Demo	2:50	11:46
P9 · Cierre	0:10	**11:56**

> **La regla que no se toca:** el demo empieza antes de los 10:00. Si una toma se estira, se recorta de P1–P2, nunca del demo.

Antes de la primera toma

- ☐ Corré `.\scripts\preflight-demo.ps1` (deja las dos clínicas y limpia los datos del video).
- ☐ Corré `.\scripts\chequear-video.ps1`: te dice en 5 segundos si falta algo. Si dice **LISTO PARA GRABAR**, los comandos de esta hoja funcionan.
- ☐ Tres ventanas: terminal en `backend/`, navegador en `http://127.0.0.1:8000/docs`, editor en la raíz.
- ☐ Abiertos: `CHANGES.md`, `openspec/changes/archive/`, `docs/skills-mcp/evidencias/`, `docs/skills-mcp/demo-mcp.md`.
- ☐ Token del MCP cargado: `$env:GITHUB_PERSONAL_ACCESS_TOKEN.Length` tiene que dar más de 0. **Nunca muestres el token**, solo el número.

> **Regla de oro de los comandos:** cada bloque empieza fijando la carpeta. Así no importa dónde quedaste en el bloque anterior: se puede pegar suelto.

P1 · C-01: lo básico andando

De qué se trata: que el proyecto levante y pueda hablar con la base de datos.

MOSTRÁS: `docker compose ps` en healthy · `alembic current` · `/health` · la carpeta archivada.

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
docker compose ps
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA\backend"
$env:DATABASE_URL="postgresql+asyncpg://tpi:tpi@localhost:5433/tpi_dev"
$env:SECRET_KEY="dev-only-change-me"
.\.venv\Scripts\python.exe -m alembic current
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
curl.exe -sS http://127.0.0.1:8000/health
Write-Output ""
Get-ChildItem openspec\changes\archive
```

DECÍS: "Arranco por lo básico: que el proyecto levante y hable con la base. Tengo una sola base con dos nombres: una para desarrollar y otra para probar. La puse en el puerto 5433 para no pelearme con un PostgreSQL que ya esté instalado. El `/health` no me dice solamente que la app arrancó: me dice que puede hablar con la base. Dos cosas que descarté: no puse un contenedor aparte para los tests, y no obligo a tener Docker para correr las pruebas. Las pruebas corren en SQLite, que es más liviano, y antes de archivar las corro también en PostgreSQL."

P2 · C-02: que cada clínica vea solo lo suyo

De qué se trata: usuarios, permisos y separación entre clínicas.

MOSTRÁS: carpeta archivada · `openspec/specs/` · `app/repositories/` + `test_architecture.py`.

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
Get-ChildItem openspec\changes\archive\2026-11-04-core-models-tenants
Get-ChildItem openspec\specs
Get-Content backend\tests\test_architecture.py -TotalCount 40
```

DECÍS: "Acá está lo importante: que cada clínica vea solo lo suyo. La clínica no la manda el cliente en el pedido: se saca del usuario que está logueado. Si alguien intenta mandar el `clinic_id` por el payload, el sistema lo rechaza. Y si pido un dato de otra clínica, me responde exactamente lo mismo que si ese dato no existiera: un 404. Así no le cuento a nadie que el dato existe en otra clínica. Los permisos funcionan al revés de lo habitual: si no tengo permiso, no puedo; y cuando me rebota, no cambia nada en la base. Las sesiones se pueden revocar de verdad: si alguien reusa un token viejo, se corta toda la familia de esa sesión. Y todas las consultas a la base viven en un solo lugar, `app/repositories`; hay un test que lo controla. Lo que descarté: no confío en el rol ni en la clínica que vienen en el token (los vuelvo a buscar en la base en cada pedido), no distingo 'no existe' de 'es de otra clínica', y no dejo consultas sueltas por ahí."

P3 · C-03: pacientes — 1:00

De qué se trata: dar de alta pacientes sin mezclarlos entre clínicas.

MOSTRÁS: carpeta archivada · spec · `services/patients.py` (ahí vive la regla del DNI).

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
Get-ChildItem openspec\changes\archive\2026-11-04-patient-management
Select-String -Path backend\app\services\patients.py -Pattern "DuplicateDni|dni_exists"
```

DECÍS: "Pacientes. El paciente nace siempre en la clínica del usuario logueado: la clínica no viene en el pedido. Si busco uno de otra clínica, me da el mismo 404 que si no existiera, así no revelo que existe. El DNI es único por clínica, no global: dos clínicas pueden atender a la misma persona, y eso se resuelve en el servicio y también con la restricción de la base. Y no se borra: se desactiva con `is_active`. Descarté el DNI global y el borrado físico. La spec y los tests cubren alta, búsqueda, edición y baja lógica."

P4 · C-04: la agenda

De qué se trata: turnos que no se pisan y estados que no se pueden saltar.

MOSTRÁS: carpeta archivada · spec · `services/appointments.py` · `repositories/appointments.py`.

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
Get-ChildItem openspec\changes\archive\2026-11-05-appointment-agenda
Select-String -Path backend\app\services\appointments.py,backend\app\repositories\appointments.py -Pattern
"VALID_TRANSITIONS|with_for_update|lock_resources"
Select-String -Path openspec\specs\appointment-agenda\spec.md -Pattern "Requirement:"
```

DECÍS: "La agenda. Un turno tiene paciente, dentista y box. La regla importante es que no se pisen dos turnos: ni por dentista ni por box. Y el borde no cuenta como choque: si uno termina 9:30 y el otro empieza 9:30, está bien. Si se superponen, 409 (conflicto). Las horas se guardan siempre en UTC; si mandás una hora sin zona horaria, te la rechazo con 422 (datos inválidos). Los estados no se pueden saltar: de `scheduled` pasás a `confirmed`; de `confirmed` a `attended` o `cancelled`; cualquier otro salto es 409. Y no me alcanza con controlarlo en Python, porque dos pedidos pueden entrar al mismo tiempo: por eso también hay un candado y una regla en PostgreSQL que lo impide a nivel base."

P5 · Skill: la que audita los changes — 1:00

De qué se trata: un procedimiento que revisa los documentos antes de programar. No programa.
MOSTRÁS: `SKILL.md` · la línea del registro · `evidencias/revision-skill-c04.md`.

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
Get-Content .agents\skills\revisor-de-agendas-de-citas\SKILL.md -TotalCount 25
Select-String -Path .atl\skill-registry.md -Pattern "revisor-de-agendas-de-citas" -Context 0,6
```

DECÍS: "La skill es un procedimiento de auditoría: revisa los documentos antes de programar, no programa. En C-04 encontró 11 observaciones; la crítica era que la spec no definía de dónde salía el `clinic_id` del turno. Las apliqué yo antes de implementar, en el commit `a0fc1cb`. La skill detecta; la decisión la tomo yo. Quedó registrada en el skill-registry con sus reglas compactas, el agente la puede invocar, y los tests verifican que las correcciones quedaron."

> **Si te preguntan el detalle (no lo digas de entrada):** fueron **1 crítica, 5 importantes y 5 menores**. La crítica, el origen del `clinic_id`. Las importantes: solapamientos incompletos (box, contención, coincidencia exacta, cancelados), datetimes con offset o naive, falta de matriz de transiciones, spec sin listado de turnos y mecanismo de concurrencia sin definir. Las correcciones quedaron en `evidencias/revision-skill-c04.md`. La skill **no** editó archivos, **no** implementó y **no** marcó tareas.

P6 · MCP: la evidencia de afuera — 0:45

De qué se trata: una conexión de solo lectura a GitHub para chequear si un patrón es estándar.
MOSTRÁS: la conexión · `get_me` · una búsqueda · después la evidencia.

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
opencode mcp list
if ($env:GITHUB_PERSONAL_ACCESS_TOKEN) {
    "token cargado, largo $($env:GITHUB_PERSONAL_ACCESS_TOKEN.Length)"
} else {
    "FALTA el token del MCP en esta ventana"
}
```

En el chat del agente:

```
Usá el MCP github: get_me
```

```
Usá search_code con: EXCLUDE USING gist tstzrange
```

DECÍS: "El MCP no es una skill: es una conexión a herramientas externas, en este caso GitHub, y es solo de lectura. Acá está conectado. Le pedí mi usuario y me confirmó que corre con mi cuenta. Después busqué cómo se evita que dos turnos se pisen en PostgreSQL: 26.976 resultados. No significa que copié el código; significa que el patrón es estándar y no me lo inventé. También busqué lo de la familia de tokens y me dio 1.282 resultados, que respaldan C-02."

> Si falla la red o el token: mostrá `docs/skills-mcp/evidencias/mcp-github-c04.md` y decí "la conexión la muestro desde la evidencia porque el token es mío y no lo expongo en cámara".

P7 · Tests — 0:20

De qué se trata: mostrar que todo lo anterior está probado.
MOSTRÁS: la suite completa.

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA\backend"
.\.venv\Scripts\python.exe -m pytest -q
```

DECÍS: "259 tests. Cada test deja la base como estaba, así que no ensucio la base de desarrollo. La misma suite la corrí en SQLite y en PostgreSQL."

P8 · Demo: que se vea funcionando — 2:50

MOSTRÁS: el script corriendo solo. **Hablá solo en estos 8 momentos.**

```
Set-Location "C:\Users\Mateows\Desktop\Parciales y Base de Datos\TPI METODOLOGIA"
.\scripts\chequear-video.ps1
.\scripts\demo-video.ps1 -SkipGrace
```

> El primer comando es opcional y tarda 5 segundos: si algo falta, te lo dice antes de que empiece la toma. Si la API está caída, el demo también te avisa y te muestra cómo levantarla.

Momento	Decís
1. `/health`	"La app y la base están conectadas."
2. login + `/me`	"Entro como admin. El token no lo muestro; solo el largo."
3. `POST /users` 201	"Creo un usuario: sale bien y la respuesta no trae contraseña."
4. mismo email 409	"Lo creo de nuevo con el mismo mail: me rebota, y no me dice de qué clínica es."
5. recepcionista 403	"Entro como recepcionista y quiero ver usuarios: no me deja."
6. PATCH usuario de otra clínica 404	"Intento tocar un usuario de otra clínica: me responde igual que si no existiera."
7. turno 201 → 409 → 201	"Creo un turno: entra. Lo piso con otro: me rebota. El que empieza justo cuando termina el anterior: entra."
8. transición 409 → naive 422 → cruce 404	"Un cambio de estado inválido: me rebota. Una hora sin zona horaria: me rebota. Y un turno de otra clínica: no lo veo."

No comentes los 20 pasos. El script ya muestra los códigos; vos explicás solo estos 8.

P9 · Cierre — 0:10

DECÍS: "Lo que falta es de C-05 para adelante, incluido el frontend. Los cuatro changes de las fases 1 y 2 están hechos, probados y archivados. No presento como terminado nada que no esté en la carpeta de archivados."

Si algo falla en cámara

Falla	Qué hacés
Docker no arranca	Cortá la toma, abrí Docker Desktop, corré `preflight` y volvé a grabar P7 y P8
El demo rebota con 409	Corré `preflight` de nuevo: limpia los datos del video
El demo dice que la API no responde	Abrí otra ventana y arrancá uvicorn (el mensaje te da el comando exacto)
El MCP no conecta	Mostrá la evidencia y decí que el token es tuyo y no lo mostrás
`pytest` no da 259	Pará: hay algo sin commitear o el entorno quedó mal cargado

> **Chequeo único:** `.\scripts\chequear-video.ps1` revisa Docker, base, API, archivos, token y scripts. Si todo da `[OK]`, los comandos de arriba no fallan por entorno.